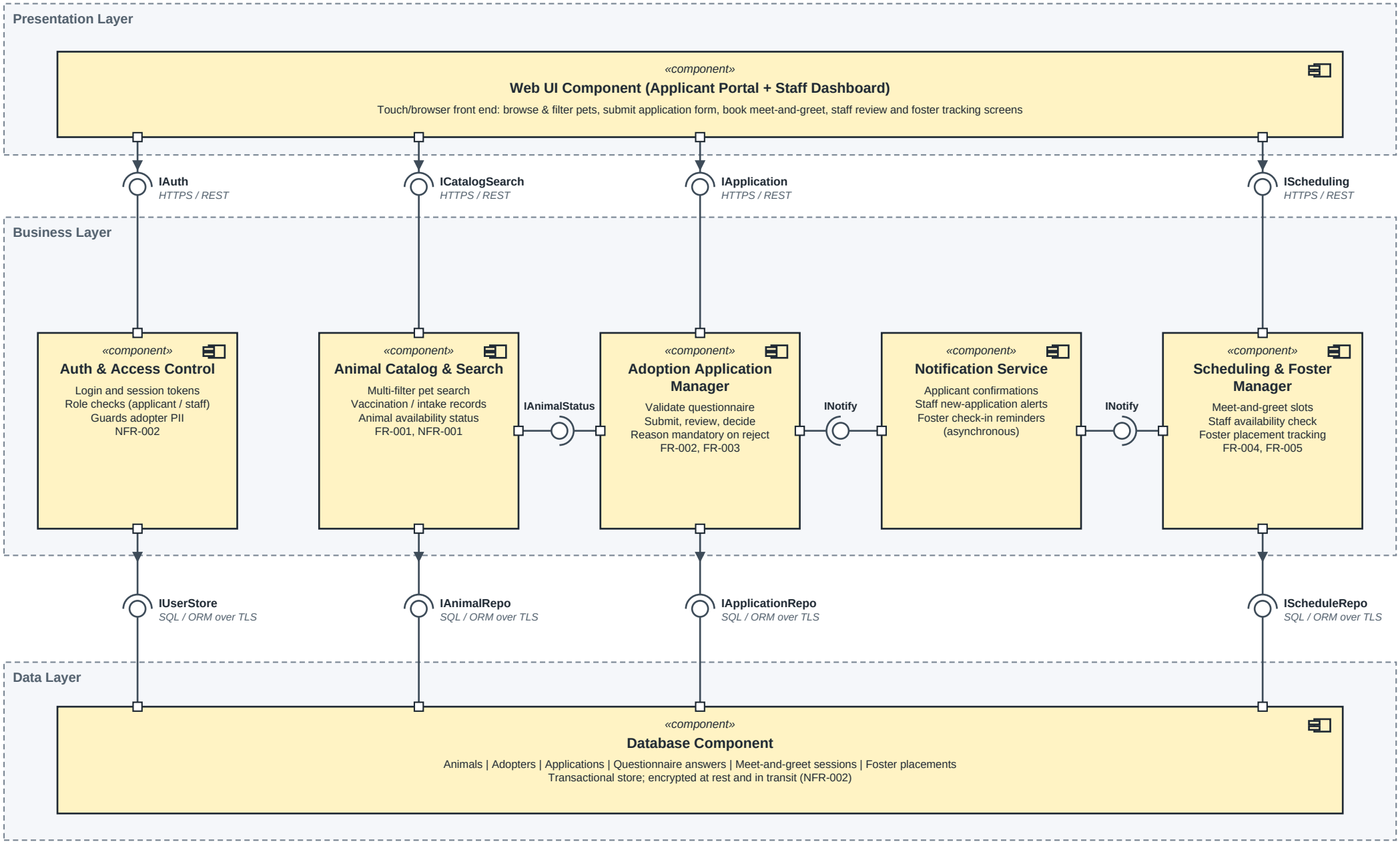
Pet Adoption & Shelter Management System - UML Component Diagram

Architectural style: Layered Architecture (Presentation / Business / Data) | Lab 3: Component Modelling



Interface legend (arrow = direction of request/data flow, requirer -> provider)

IAuth: Web UI -> Auth (credentials in, role token out)
ICatalogSearch: Web UI -> Catalog (filters in, animal list out)
IApplication: Web UI -> Application Manager (form in, status out)
IScheduling: Web UI -> Scheduling (slot request in, booking out)
IAnimalStatus: App Manager -> Catalog (check / set animal status)
INotify: App Manager and Scheduling -> Notification (async message)
IUserStore / IAnimalRepo / IApplicationRepo / IScheduleRepo: business -> Database
Ball = provided interface; socket = required interface
Small square = port on component boundary

Lab 3 - Architectural Justification

Shana Susan Varghese | PES University | Pet Adoption & Shelter Management System

Architecture Selection: We chose **Layered Architecture** for the **Pet Adoption & Shelter Management System**.

Architectural Choice

A three-layer style: a **Presentation layer** (Web UI for applicants and staff), a **Business layer** (Auth & Access Control, Animal Catalog & Search, Adoption Application Manager, Notification Service, Scheduling & Foster Manager) and a **Data layer** (Database Component). Requests travel only downward through provided/required interfaces, e.g. IApplication, IAnimalStatus, IApplicationRepo.

Style Comparison

Style	Fit for this scenario	Verdict
Layered	Rule-heavy workflows (FR-001 to FR-005), one shelter, one data store, moderate traffic.	**Selected**
Microservices	Independent scaling and deployment, but adds network latency, distributed transactions and operations effort a small shelter team cannot justify.	Over-engineered
Client-Server	Simple and centrally controlled, but says nothing about structuring the server logic; a single monolith would mix rules, search and data access.	Too coarse

Two Reasons

1. The system is driven by business rules that must never be bypassed. An application is accepted only with a complete questionnaire (FR-002), a rejection needs a recorded reason (FR-003), a meet-and-greet is booked only when a staff slot is free (FR-004), and only "Ready for Adoption" animals can be applied for (FR-001). Layered design keeps every rule in one Business-layer component, so the UI cannot skip validation, and each requirement maps to a single component.

2. Animal status must stay consistent in one shared data store. In the Submit Adoption Application use case (alternate flow A2) two applicants can race for the same animal. A single Data layer with transactional updates through IAnimalStatus and IAnimalRepo lets the second submission be rejected cleanly. Microservices would need distributed transactions for the same guarantee.

Security Advantage

NFR-002 requires adopter data to be visible only to authenticated staff roles. In a layered design every path to personal data goes UI, then Business, then Data; the Database Component accepts calls only from Business-layer components, never from the UI. Auth & Access Control issues role-based tokens that each business component verifies, so unauthenticated calls return 403, while TLS in transit and encryption at rest protect the stored data. There is one controlled choke point to secure and audit.

Performance Benefit

NFR-001 asks for catalog pages under 200 ms. Search is isolated in the Animal Catalog & Search component behind ICatalogSearch and IAnimalRepo, so indexed multi-attribute queries (species, age, breed, good with kids) and caching of the read-mostly catalog can be tuned in one place without touching the adoption workflow. Slow work such as emails and reminders goes through the asynchronous Notification Service, so it never delays a user's response.

Trade-off accepted: each extra layer adds a call hop and scaling is per application rather than per component; for a single shelter's traffic this cost is small compared with the gains in simplicity, rule enforcement and data consistency.